

Object-Oriented Programming Team Project

Campus Event Management System — Java

A collaborative Java project focused on object-oriented design, Git/GitHub teamwork, system integration, documentation, UML, and live demonstration.

Primary Goals	Design with objects • Build in parallel • Integrate continuously • Explain the architecture
Final Deliverables	Working Java code • GitHub repository • Technical documentation • UML diagrams • Live team demo
Author	Homi Bodhanwala

Project premise: Your team will design and implement a console-based campus event management system in Java. The project is intentionally large enough that multiple developers must coordinate class interfaces, develop features on separate branches, review each other's work, and integrate those components into one coherent application.

1. Project Overview

In this project, your team will design and implement a **Campus Event Management System** in Java. Colleges regularly organize workshops, guest lectures, club meetings, career fairs, study sessions, and social events. Students need a way to discover and register for events, while organizers need a way to create events, manage attendance, and view information about their events.

This is not simply a programming assignment. It is an introduction to building a moderately sized software system as a team. You will practice designing the system before implementation, dividing responsibilities across classes and packages, developing in parallel with Git and GitHub, integrating code written by multiple developers, testing the complete application, documenting the architecture, and presenting the finished system.

2. Learning Objectives

- Design Java classes with clear responsibilities and appropriate public/private interfaces.
- Use constructors, encapsulation, composition, inheritance, abstract classes or interfaces where appropriate, method overriding, and polymorphism.
- Use Java object references appropriately and avoid unnecessary duplication of data.
- Use Java Collections such as `ArrayList` where appropriate.
- Store, search, add, remove, and relate collections of objects.
- Organize a program across multiple classes and packages without placing application logic in `main()`.
- Use exception handling appropriately for input, file operations, and other recoverable errors.
- Use Git branches and Pull Requests so multiple developers can work in parallel.
- Integrate independently developed components and resolve merge or interface conflicts.
- Create UML diagrams and technical documentation that accurately represent the final implementation.
- Explain technical and design decisions during a live demonstration.

3. The Scenario

Your college wants a simple application for managing campus events. The system will contain information about **students**, **events**, **organizers**, and **registrations**.

Students

Students have information such as student ID, name, email, and major. Students should be able to view events and register for events.

Events

Events have an event ID, name, description, date, time, location, and maximum capacity. Different types of events may contain additional information. Examples include academic events, club events, social events, and career events.

Organizers

An organizer creates and manages events. An organizer might have an organizer ID, name, email, and department or organization.

Registrations

The system must track which students are registered for which events. A registration might contain references to a student and event, a registration date, and registration status. Your team must determine the appropriate relationships between these objects.

4. Required Application Functionality

The final program must provide a menu-driven console interface. Your exact menu may differ, but the program must make the major system operations easy to access.

```
=====
      CAMPUS EVENT MANAGEMENT SYSTEM
=====

1. Add Student
2. View Students
3. Create Event
4. View Events
5. Search Events
6. Register Student for Event
7. Cancel Registration
8. View Student Registrations
9. View Event Attendees
10. View Organizers
11. System Reports
0. Exit

Enter selection:
```

5. Student Management

- Add a student.
- Find a student by student ID.
- Display an individual student.
- Display all students.
- Prevent duplicate student IDs from being added.

6. Event Management

- Create an event.
- Find an event using its unique ID.
- Display an individual event.
- Display all events.
- Search for events by event name at minimum.
- Optionally support searches by event type, date, organizer, or location.
- Track maximum capacity and prevent registrations beyond that capacity.

```
Event: Introduction to Robotics
Date: October 15
Location: Engineering Building 201
Capacity: 30
Registered: 24
Spaces Remaining: 6
```

7. Different Types of Events

The application must support multiple event types. Your team should decide which information belongs in the base class and which information belongs in derived classes.

```
Event
|
+-- AcademicEvent
|
+-- ClubEvent
```

```
|  
+-- CareerEvent  
|  
+-- SocialEvent
```

In Java, the base Event class may be a regular class or an abstract class depending on your design. Derived classes use extends.

```
public abstract class Event {  
    private int eventId;  
    private String name;  
    private int capacity;  
  
    public abstract void displayDetails();  
}  
  
public class CareerEvent extends Event {  
    private String companyName;  
  
    @Override  
    public void displayDetails() {  
        // Display common event information  
        // plus CareerEvent-specific information.  
    }  
}
```

Important: Do not create subclasses simply because the assignment asks you to demonstrate inheritance. Your hierarchy should represent a meaningful **is-a** relationship. A CareerEvent **is an** Event.

8. Polymorphism

Your application must contain at least one meaningful use of polymorphism. Java dynamically dispatches overridden instance methods, so a base-class reference can refer to an object of a derived class.

```
Event event = new CareerEvent();  
  
// Java calls the CareerEvent implementation.  
event.displayDetails();
```

Your documentation must identify where polymorphism is used and explain why it improves the design. Use `@Override` when overriding methods.

9. Event Registration

Students must be able to register for events. Before completing a registration, the system should verify that:

- The student exists.
- The event exists.
- The student is not already registered for the event.
- The event has available capacity.

The system must also allow an existing registration to be canceled.

```
Registration successful!  
  
Student: Maria Lopez  
Event: Introduction to Robotics
```

10. Viewing Registrations

The application must support viewing registrations from both directions.

View events for a student

Student: Maria Lopez

Registered Events:

1. Introduction to Robotics
2. Resume Workshop
3. Computer Science Club Meeting

View students attending an event

Event: Introduction to Robotics

Registered Students:

1. Maria Lopez
2. David Chen
3. Sarah Johnson

Think carefully about which object owns or manages these relationships. Java variables that refer to objects contain references to those objects; you should not create duplicate Student or Event objects merely to represent a registration.

11. Organizer Management

- Add organizers.
- Find and display organizers.
- Associate an organizer with an event.
- View events managed by a particular organizer.

Your team must decide whether an event directly stores an Organizer reference, stores an organizer ID, or uses another relationship. Be prepared to explain the tradeoffs.

12. Reports

The application must generate at least **three useful reports**. Examples include:

- Events with available capacity.
- Events that are full.
- Number of students registered for each event.
- Events organized by a particular organizer.
- Events of a particular type.
- Most popular events.
- Students registered for multiple events.

13. Java Collections

Use Java Collections where they improve the design. For example, the system may use `ArrayList<Student>`, `ArrayList<Event>`, or other appropriate collection types.

```
private ArrayList<Student> students;  
private ArrayList<Event> events;  
private ArrayList<Registration> registrations;
```

Your team should be able to explain why a particular collection was chosen. Do not select a collection solely because it appears in this example.

14. Data Persistence

The program must save important application data so that information is not lost when the program exits. When the program starts, it should load previously saved information when available; when it exits, it should save the current information.

Your team must determine what files are needed, what information must be saved, and how relationships between objects will be reconstructed after loading. You may use appropriate Java file I/O classes and should document your file format.

15. Exception Handling and Input Validation

The application should handle incorrect input and recoverable errors gracefully rather than crashing. Use validation and Java exception handling where appropriate.

- Duplicate student or event IDs.
- Student or event IDs that do not exist.
- Registration for a full event.
- Duplicate registrations.
- Invalid menu selections or malformed user input.
- Missing or invalid data files.
- File I/O errors.
- Attempting to cancel a registration that does not exist.

```
try {  
    // Perform an operation that may fail.  
} catch (IOException e) {  
    // Handle the error appropriately.  
}
```

Do not use exceptions as a replacement for ordinary validation. Use them when they are appropriate to the failure being handled.

16. Object-Oriented Design Expectations

Your grade will not be based only on whether the program works. The **design** of the program is an important part of the project.

- What classes should exist, and what responsibility does each class have?
- Which fields should be private?
- Which classes should know about other classes?
- Where should inheritance be used? Where is composition a better choice?
- Would an interface or abstract class improve the design?
- Which methods belong to which classes?
- Which objects own or manage collections of other objects?
- Where does polymorphism make the design simpler or more extensible?

Avoid the “God class”: Do not create one enormous class that performs most of the work while the remaining classes merely hold data.

17. Possible Class Structure

You are **not required** to use exactly this design. It is provided as a starting point; part of the assignment is deciding how the classes should interact.

Student
Organizer

```
Event
  AcademicEvent
  ClubEvent
  CareerEvent
  SocialEvent
```

```
Registration
EventSystem
FileManager
```

18. UML Class Diagram

Before completing the implementation, your team must create a UML class diagram representing the system design. The final diagram must match the final implementation.

- Classes.
- Important fields.
- Important methods.
- Inheritance.
- Associations.
- Composition or aggregation where appropriate.
- Interfaces or abstract classes where appropriate.
- Relationships between major classes.

[illegible]

The example above is intentionally incomplete. If your implementation changes during development, update the UML diagram.

19. Team Development Using Git and GitHub

This is a **team software development project**. Your team must use Git and GitHub throughout the project. Git should not simply be used to upload the final program. Your repository history should demonstrate that multiple team members actively contributed over time.

20. Suggested Java Project Structure

```
CampusEventSystem/
|
+-- README.md
+-- docs/
|   +-- UML.pdf
|   +-- DESIGN.md
|
+-- src/
|   +-- app/
|   |   +-- Main.java
|   |
|   +-- model/
|   |   +-- Student.java
|   |   +-- Organizer.java
|   |   +-- Event.java
|   |   +-- AcademicEvent.java
|   |   +-- ClubEvent.java
|   |   +-- CareerEvent.java
|   |   +-- SocialEvent.java
|   |   +-- Registration.java
|   |
|   +-- service/
|   |   +-- EventSystem.java
|   |
|   +-- persistence/
|   |   +-- FileManager.java
|   |
+-- data/
|   +-- students.txt
|   +-- organizers.txt
|   +-- events.txt
|   +-- registrations.txt
```

Your team may organize packages differently if there is a clear reason. Use package names consistently and keep related classes together.

21. Parallel Development Workstreams

The project is intentionally structured so several students can work simultaneously. A four-person team might initially divide work as follows:

Developer	Primary Workstream	Example Responsibilities
A	Student Management	Student class, creation, searching, display
B	Event System	Event hierarchy, subclasses, searching, display
C	Registration System	Registration, cancellation, duplicate/capacity checks
D	Organizer & Persistence	Organizer, file loading/saving, persistence

This is only an example. Teams should agree on public class interfaces and package responsibilities early so workstreams can proceed in parallel without repeatedly changing another developer's implementation.

22. Git Workflow

Maintain a working main branch. New work should normally be performed on separate feature branches.

```
main

feature/student-management
feature/event-management
```

```
feature/registration
feature/file-storage
feature/reports
```

When a feature is ready, push the branch and create a Pull Request into main. A teammate should review the change before it is merged.

23. Git Expectations

Each team member is expected to clone the repository, create and work on branches, make meaningful commits, push branches, pull teammates' changes, participate in Pull Requests, review code, and resolve merge conflicts when necessary.

Good commit messages

```
Add capacity checking to event registration
Implement CareerEvent displayDetails override
Fix duplicate registration validation
```

Avoid

```
stuff
changes
final
final2
actually-final
```

24. Integration Is a Team Responsibility

Do not wait until the final day to combine everyone's code. Integrate work continuously throughout the project.

```
Developer A --+
                |
Developer B --+--> Pull Requests --> main --> Working Application
                |
Developer C --+
                |
Developer D --+
```

After a Pull Request is merged, everyone should update their local repository. Integration problems are expected; learning to identify and resolve them is part of the project.

25. README Documentation

- **Project Overview:** What does the application do?
- **Team Members:** Each team member and their primary responsibilities.
- **Features:** Major capabilities of the program.
- **Class Design:** Major classes and responsibilities.
- **Packages:** Explain the purpose of major packages.
- **Building:** Complete compilation/build instructions.
- **Running:** How to start and use the program.
- **Data Storage:** How data is saved and loaded.
- **Known Limitations:** Important limitations or unfinished functionality.

26. Technical Design Documentation

- Major classes and their responsibilities.
- Relationships between classes.

- Inheritance hierarchy.
- Use of composition.
- Use of abstract classes or interfaces, if applicable.
- Use of polymorphism.
- Important design decisions and tradeoffs.
- How data is stored and reconstructed.
- How the team divided the project.
- Significant technical challenges and how they were resolved.

Do not simply paste source code into the documentation. Explain the design.

27. Source Code Requirements

- Compile without errors and run without crashing during normal use.
- Use appropriate Java classes and packages.
- Use meaningful class, method, field, and variable names.
- Keep fields private unless there is a strong reason otherwise.
- Use constructors and methods to maintain valid object state.
- Use `@Override` when overriding methods.
- Use Java Collections appropriately.
- Avoid unnecessary static/global-style state and duplicated data.
- Use consistent formatting and appropriate comments.
- Keep `main()` focused primarily on starting and coordinating the application; application logic belongs in appropriate classes.

28. Final Deliverables

1. Complete Source Code

A fully working Java application. The submitted version must compile and run.

2. GitHub Repository

Complete Git history demonstrating team participation, including source code, README, documentation, UML, and sample data.

3. UML Class Diagram

A final diagram representing the classes and relationships in the completed implementation.

4. Technical Documentation

Architecture, object relationships, design decisions, persistence strategy, package organization, and development approach.

5. Live Team Demonstration

A live presentation in which every team member participates.

29. Final Presentation and Demonstration

Project Introduction

- What you built.
- What problem it solves.

- Major capabilities.

Architecture

Show the UML diagram and explain the most important classes and how they interact. Do not simply read the diagram; explain **why** the system was designed this way.

OOP Concepts

Show examples from your program demonstrating encapsulation, composition, inheritance, method overriding, and polymorphism. If you used an interface or abstract class, explain why.

Live Demonstration

- Creating or viewing students.
- Creating or viewing events.
- Registering a student for an event.
- Preventing an invalid registration.
- Viewing event attendees.
- Demonstrating different event types.
- Demonstrating at least one report.
- Demonstrating saved/persisted data.

Git Collaboration

Show the GitHub repository, including branches, commit history, Pull Requests, and contributions from different team members. Explain how the work was divided.

Lessons Learned

- A technical challenge the team encountered.
- A collaboration or Git challenge.
- Something the team would design differently if starting again.

30. Individual Accountability

Although this is a team project, every student is responsible for understanding the complete application. During the demonstration, the instructor may ask **any team member** questions about the program.

“Why is this relationship represented using an object reference?”

“Why is Event abstract?”

“Why did you override this method?”

“What happens when a CareerEvent object is referenced by an Event variable?”

“Why did you use inheritance here instead of composition?”

“Why did you choose ArrayList for this collection?”

“How does the application prevent duplicate registrations?”

“How does this data get reconstructed when the application starts?”

“Why is this exception caught here?”

You should be able to explain your team's code even if another team member originally implemented that section.

31. Minimum Project Requirements Checklist

☐ Multiple interacting classes	☐ Encapsulation
☐ Constructors	☐ Composition
☐ Inheritance	☐ Method overriding
☐ Polymorphism	☐ Java Collections
☐ Searching	☐ Adding and removing data
☐ Relationships between objects	☐ File input/output
☐ Input validation	☐ Exception handling
☐ Multiple Java source files	☐ Packages
☐ UML class diagram	☐ Technical documentation
☐ README	☐ Git branches
☐ Meaningful commits	☐ Pull Requests
☐ Contributions from every team member	☐ Fully integrated working application
☐ Live demonstration	

32. Stretch Goals

- Event waitlists and registration deadlines.
- Event categories or event cancellation.
- Student schedules and prevention of scheduling conflicts.
- Organizer authentication.
- Sorting events by date or popularity.
- Advanced searching and filtering.
- Attendance/check-in tracking.
- Exporting reports.
- More sophisticated persistence.
- Additional event subclasses.
- Use of additional Java interfaces where they improve the design.

Complete the core requirements before attempting stretch goals.

33. Final Perspective

The objective is not to see how many lines of Java your team can write. The objective is to take a moderately sized problem and answer three important questions:

How should we model this problem using objects?

How should those objects interact?

How can several developers build those pieces independently and successfully combine them into one working system?

A successful project should demonstrate both **good object-oriented design** and **effective software engineering teamwork**.